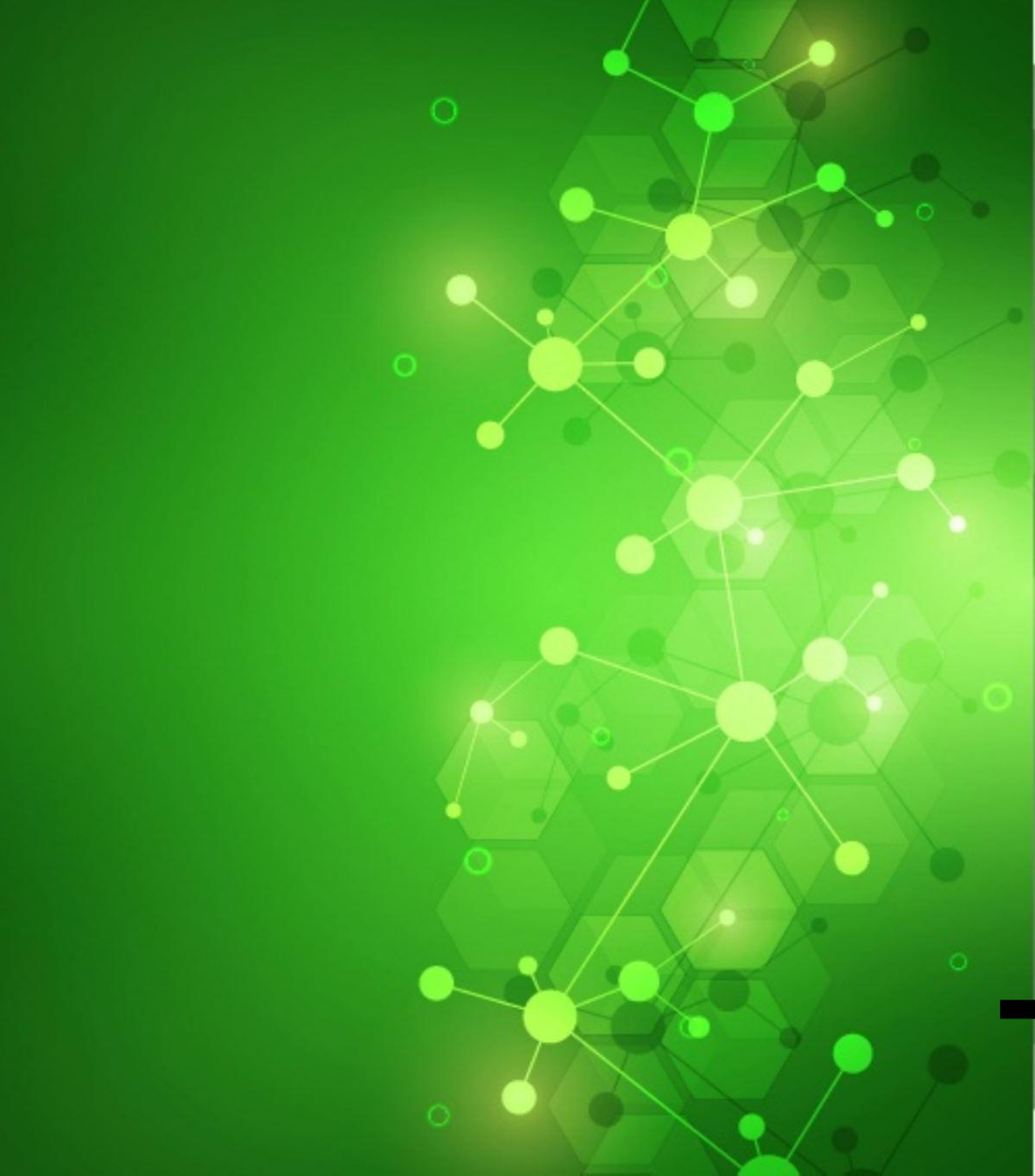




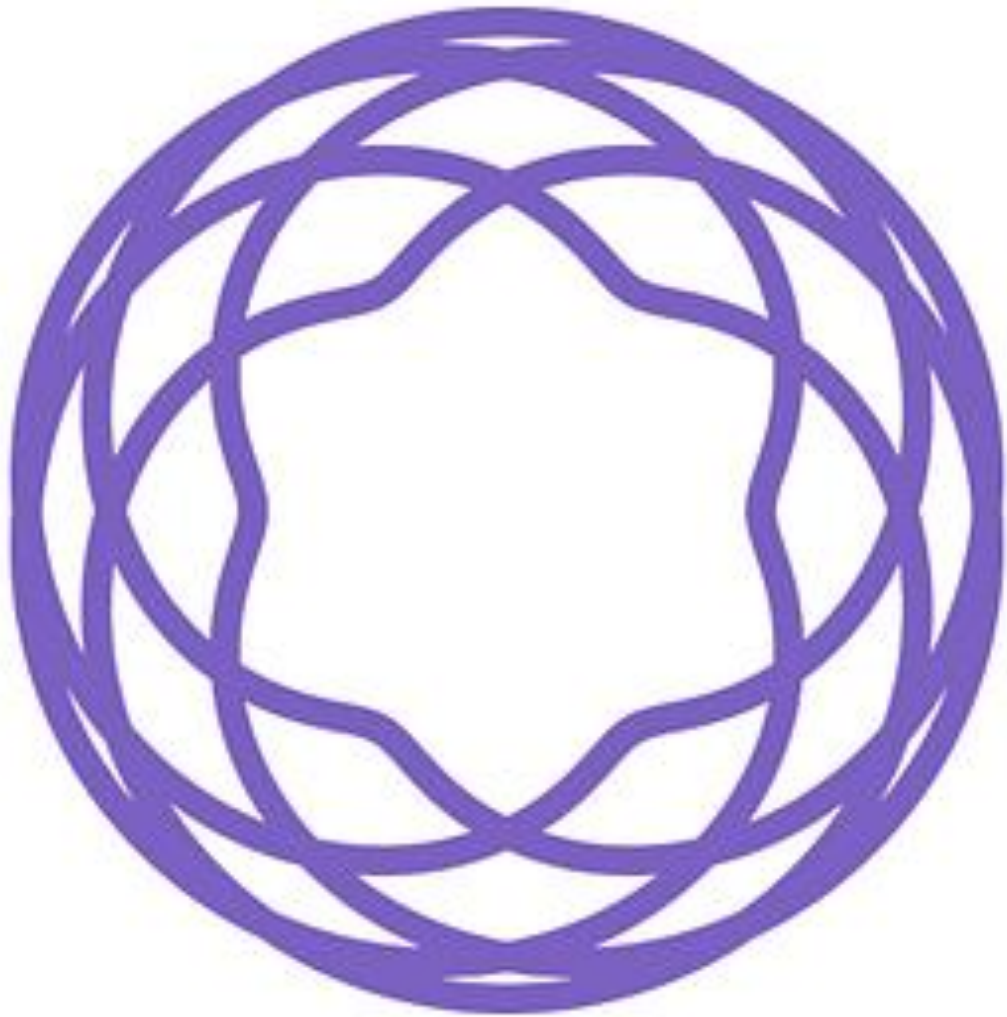
Asztali és mobil alkalmazások fejlesztése és tesztelése tantárgy – Mobil alkalmazások rész

13. A

Tanárok: Dr. Koncz Adrienn és
Simon Martin

2026.03.12.





+

React
Navigation

Fejléc
(header)



Header (fejléc) gombok

Most, hogy már tudjuk, hogyan szabhatjuk
testre a fejlécek megjelenését, tegyük őket
interaktívvá!



Farsang 🎭

A farsang a vidámság, a jelmezek és az ünneplés
időszaka. Fedezd fel a hagyományokat, nézz
meg hangulatos képeket, és böngéssz az

Home

Info

Home Screen

Gomb hozzáadása a fejléchez

<https://reactnavigation.org/docs/header-buttons>

- A fejléc egyik leggyakoribb használati módja, hogy a cím bal vagy jobb oldalán lévő gombra kattintunk.
- Adjunk hozzá egy gombot a fejléc jobb oldalához:

```
const MyStack = createNativeStackNavigator({
  screens: {
    Home: {
      screen: HomeScreen,
      options: {
        headerRight: () => (
          <Button onPress={() => alert('This is a button!')}>Info</Button>
        ),
      },
    },
  },
});
```

headerLeft is működik, akkor a gomb a bal oldalra kerül

Gomb hozzáadása a fejléchez

- Amikor így definiáljuk a gombot, nem tudjuk elérni vagy frissíteni benne a képernyőkomponens állapotát (state).
- Ez elég fontos, mert gyakran szeretnénk, hogy a fejlécben lévő gombok interakcióba lépjenek azzal a képernyővel, amelyhez a fejléc tartozik.
- A következőkben megnézzük, hogyan lehet ezt megoldani.

```
const MyStack = createNativeStackNavigator({
  screens: {
    Home: {
      screen: HomeScreen,
      options: {
        headerRight: () => (
          <Button onPress={() => alert('This is a button!')}>Info</Button>
        ),
      },
    },
  },
});
```



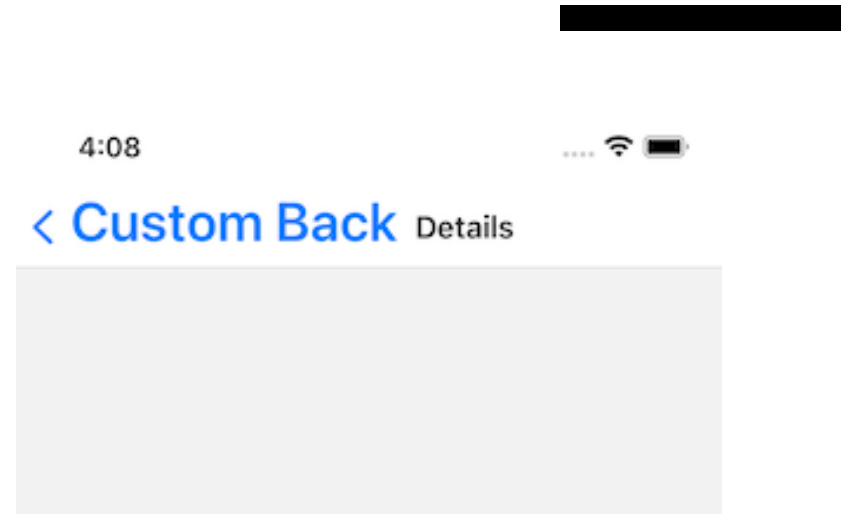
A fejléc interakciója a képernyő komponensével

- Ahhoz, hogy a fejlécben lévő gombok kapcsolatba lépjenek a képernyő állapotával (state), használhatjuk a `navigation.setOptions` függvényt a képernyő komponensén belül: 
- Így frissítjük a `headerRight` elemet egy olyan gombbal, amelynek van egy `onPress` kezelője.
- Mivel ez a komponensen belül van definiálva, hozzáfér a komponens állapotához, és módosítani is tudja azt.

```
function HomeScreen() {  
  const navigation = useNavigation();  
  const [count, setCount] = React.useState(0);  
  
  React.useEffect(() => {  
    // Use `setOptions` to update the button that we previously specified  
    // Now the button includes an `onPress` handler to update the count  
    navigation.setOptions({  
      headerRight: () => (  
        <Button onPress={() => setCount((c) => c + 1)}>Update count</Button>  
      ),  
    });  
  }, [navigation]);  
  
  return <Text>Count: {count}</Text>;  
}  
  
const MyStack = createNativeStackNavigator({  
  screens: {  
    Home: {  
      screen: HomeScreen,  
      options: {  
        // Add a placeholder button without the `onPress` to avoid flicker  
        headerRight: () => <Button>Update count</Button>,  
      },  
    },  
  },  
});
```

A vissza gomb testreszabása

- A vissza gomb automatikusan megjelenik egy stack navigatorban, amikor van egy másik képernyő, ahová vissza lehet lépni.
- A native stack navigator platformspecifikus alapértelmezéseket használ ehhez a vissza gombhoz. *Régebbi iOS-verziókon például előfordulhat, hogy a gomb mellett egy felirat is megjelenik, amely az előző képernyő címét mutatja, ha van rá elegendő hely.*
- A vissza gombot többféle opcióval is testre szabhatod, például:
 - **headerBackTitle:** a vissza gomb feliratának módosítása (iOS)
 - **headerBackTitleStyle:** a vissza gomb feliratának stílusa
 - **headerBackIcon:** egyéni vissza gomb ikon



```
const MyStack = createNativeStackNavigator({
  screens: {
    Home: {
      screen: HomeScreen,
      options: {
        headerBackTitle: 'Custom Back',
        headerBackTitleStyle: { fontSize: 30 },
      },
    },
  },
});
```



A vissza gomb testreszabása

- Ha a fenti opcióknál nagyobb mértékben szeretnéd testre szabni, használhatod a headerLeft opciót, amellyel saját komponenst jeleníthetsz meg a fejléc bal oldalán.
- A headerLeft opció egy React komponenst vár, amelynek segítségével felülírhatod az onPress működését, vagy akár teljesen le is cserélheted a gombot.
- A részletekért nézd meg az API-dokumentációt:
 - <https://reactnavigation.org/docs/native-stack-navigator/#headerleft>

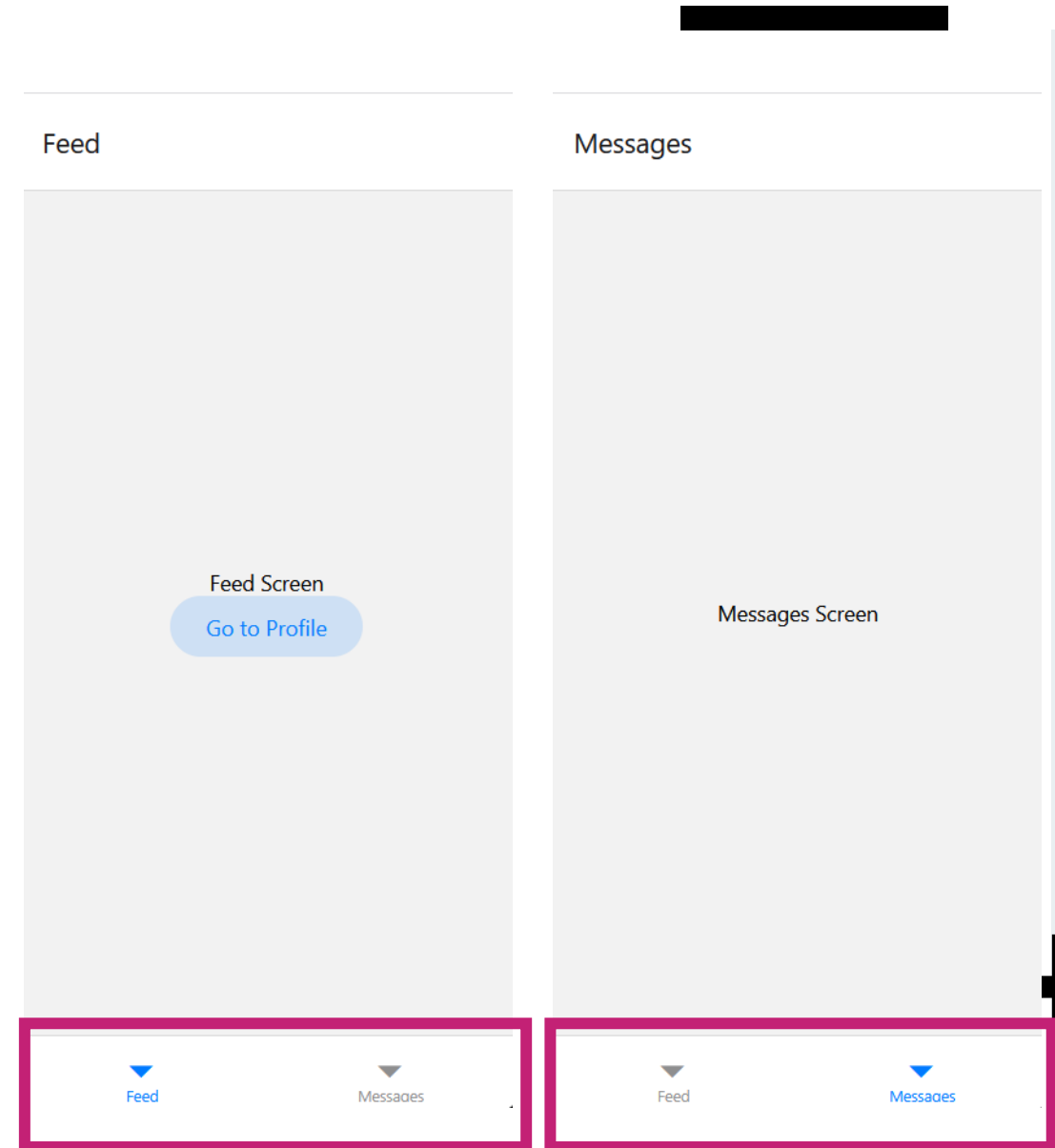


Navigátorok egymásba ágyazása

A navigátorok egymásba ágyazása (nesting navigators) azt jelenti, hogy egy navigátort egy másik navigátor egyik képernyőjén belül jelenítünk meg, például:

```
const HomeTabs = createBottomTabNavigator({
  screens: {
    Feed: FeedScreen,
    Messages: MessagesScreen,
  },
});

const RootStack = createNativeStackNavigator({
  screens: {
    Home: {
      screen: HomeTabs,
      options: {
        headerShown: false,
      },
    },
    Profile: ProfileScreen,
  },
});
```



Navigátorok egymásba ágyazása

Ebben a példában egy tab navigátor (HomeTabs) van beágyazva egy stack navigátorba (RootStack) a Home képernyő alatt.

A struktúra így néz ki:

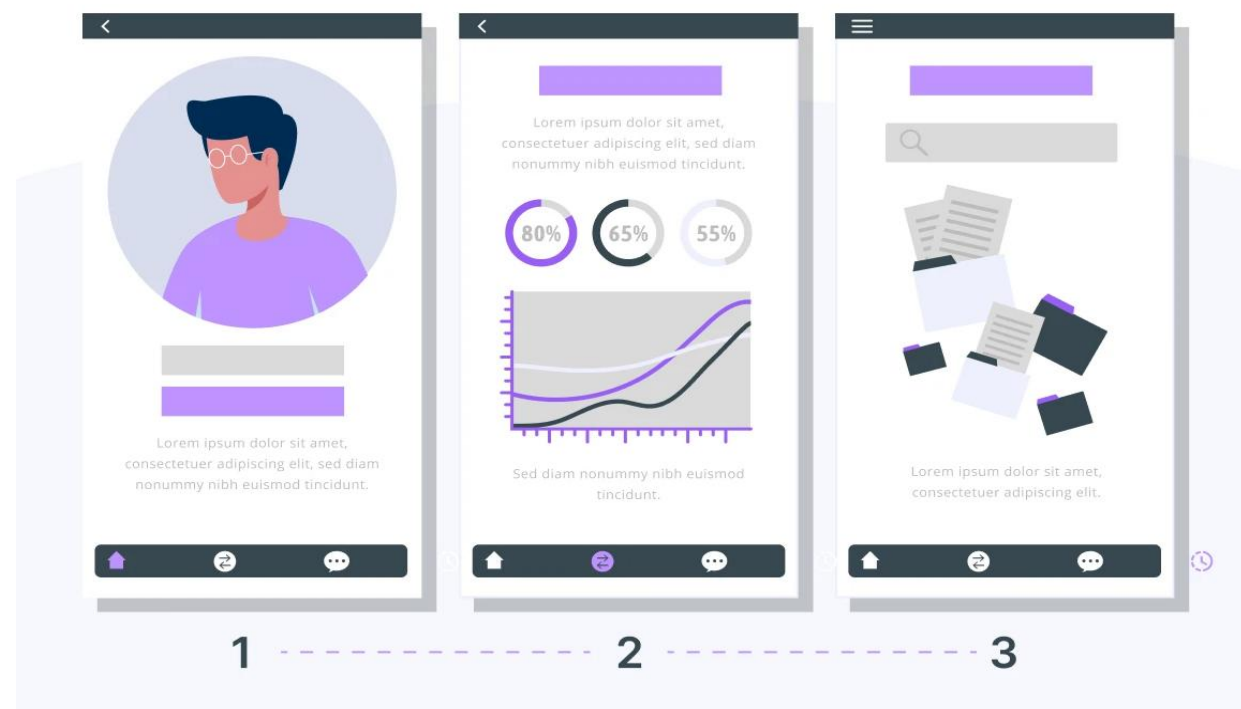
- RootStack (Stack navigátor)
 - HomeTabs (Tab navigátor)
 - Feed (képernyő)
 - Messages (képernyő)
 - Profile (képernyő)

```
const HomeTabs = createBottomTabNavigator({
  screens: {
    Feed: FeedScreen,
    Messages: MessagesScreen,
  },
});

const RootStack = createNativeStackNavigator({
  screens: {
    Home: {
      screen: HomeTabs,
      options: {
        headerShown: false,
      },
    },
    Profile: ProfileScreen,
  },
});
```

A navigátorok egymásba ágyazásának hatása a működésre

Amikor navigátorokat egymásba ágyazunk, néhány fontos dolgot érdemes szem előtt tartani.



1) Minden navigátornak saját navigációs előzménye van

- Például amikor egy stack navigátor egyik képernyőjén megnyomod a vissza gombot, amely egy másik navigátoron belül van beágyazva, akkor a visszalépés a képernyőhöz legközelebbi ős navigátor előző képernyőjére történik.
- Ha ez az első képernyő a beágyazott stackben, akkor a vissza gomb megnyomása a szülő navigátor előző képernyőjére visz.



1) Minden navigátornak saját navigációs előzménye van - Példa

Instagram



- Az Instagram alsó tab navigációt használ:
 - Home
 - Reels
 - Messages
 - Search
 - Profile
- Ha a Home tabban **Feed** → **Post** → **Comments** útvonalon haladsz.
- Majd átváltasz a Profile tabra, ott: **Profile** → **Followers**

- Amikor visszamész a Home tabra, akkor nem a Feedhez, hanem a Comments képernyőhöz térsz vissza.
- Miért? → Mert minden tab saját stack history-val rendelkezik.
- Tehát:
 - Home Tab
 - Feed
 - Post
 - Comments
 - Profile Tab
 - Profile
 - Followers
- Mindkettő külön history.



2) Minden navigátornak saját opciói vannak

- Például ha egy gyermek navigátorban lévő képernyőn megadsz egy **title** opciót, az nem fogja befolyásolni a szülő navigátorban megjelenő címet.
- Ha a szülő navigátor opcióit a gyermek navigátor aktív képernyője alapján szeretnéd beállítani, akkor nézd meg a *screen options with nested navigators* részt.
 - <https://reactnavigation.org/docs/screen-options-resolution/#setting-parent-screen-options-based-on-child-navigators-state>



2) Minden navigátornak saját opciói vannak - Példa



- A Gmailben van egy drawer menu (bal oldali menü) és azon belül stack navigation
- Például:
 - Inbox
 - Email list
 - Email detail
- Az Email detail képernyő címe lehet: Subject: Meeting tomorrow
- De a drawer fejlécében még mindig ez van: Inbox
- Miért? → Mert a stack navigator címe nem változtatja meg a drawer navigator címét.



3) Minden képernyőnek saját paraméterei (params) vannak



- Azok a paraméterek, amelyeket egy beágyazott navigátor képernyőjének adsz át, az adott képernyő **route** objektumában lesznek elérhetők.
- Ezek nem érhetők el közvetlenül a szülő vagy más gyermek navigátor képernyőiből.
- Ha egy gyermek képernyőnek hozzá kell férnie a szülő képernyő paramétereihöz, akkor használhatod a React Contextet, hogy elérhetővé tedd azokat a gyermek komponensek számára.
 - <https://react.dev/reference/react/useContext>



3) Minden képernyőnek saját paraméterei (params) vannak - Példa

Webshop (pl. Amazon)

- Képzeld el egy webshop navigációt:
 - Tabs
 - ☐ Home
 - ☐ Categories
 - ☐ Cart
- A Categories tabon belül:
 - Categories
 - ☐ Category: Shoes
 - ☐ Product: Nike Air Max
- Amikor a **Product screen** megnyílik, paramétert kap: **productId: 123**
- Ez a paraméter csak a **Product screen route**-jában lesz elérhető.
- A **Categories screen** nem látja automatikusan ezt a paramétert.
- Ezért ha több képernyőnek kell ugyanaz az adat gyakran Context / global state használata szükséges.



4) A navigációs műveleteket az aktuális navigátor kezeli

- A navigációs műveletek először az aktuális navigátorhoz kerülnek.
- Ha az nem tudja kezelni őket, akkor továbbterjednek a szülő navigátor felé.
- Példa: ha egy beágyazott képernyőn meghívod a `goBack()` függvényt:
 - először a beágyazott navigátorban lép vissza
 - ha már az első képernyőn vagy, akkor a szülő navigátorban lép vissza



4) A navigációs műveleteket az aktuális navigátor kezeli - Példa

Facebook



- Facebook navigáció:
 - Tabs
 - ☐ Home
 - ☐ Watch
 - ☐ Marketplace
- A Home tabban:
 - Feed
 - ☐ Post
 - ☐ Comments
- Ha a [Comments screenen](#) megnyomod a [back gombot](#), akkor visszamegy [Post screenre](#)
- Ha újra megnyomod, akkor visszamegy [Feed screenre](#)
- Ha ott is megnyomod, akkor [kilép a tab historyból](#)
- Ez azért van, mert [Comments](#) → [Post](#) → [Feed](#) mind ugyanabban a stackben van.



5) A navigátor specifikus metódusok elérhetők a beágyazott navigátorok képernyőin

- Például ha egy stack navigátor van egy drawer navigátor belsejében, akkor a drawer navigátor metódusai (`openDrawer`, `closeDrawer`, `toggleDrawer` stb.) elérhetők lesznek a stack navigátor képernyőinek `navigation` objektumában.
- Hasonlóképpen, ha egy tab navigátor van egy stack navigátor belsejében, akkor a tab képernyők `navigation` objektumában elérhetők lesznek a stack metódusai, például a `push` és a `replace`.
- Ha a szülő navigátorból szeretnél műveleteket küldeni egy beágyazott gyermek navigátornak, használhatod a `navigation.dispatch` metódust:

```
navigation.dispatch(DrawerActions.toggleDrawer());
```



5) A navigátor specifikus metódusok elérhetők a beágyazott navigátorok képernyőin - Példa

YouTube



- YouTube navigáció:
 - Drawer
 - ☐ Home
 - ☐ Subscriptions
 - ☐ Library
- A Home-on belül:
 - Home
 - ☐ Video
 - ☐ Comments
- Ha a **Video screenen** vagy, még mindig meg tudod nyitni a **drawer menüt**.
- Tehát egy stack screen is hívhatja a **navigation.openDrawer()** -t
- Ez azért működik, mert **a drawer a parent navigator**.



6) A beágyazott navigátorok nem kapják meg a szülő eseményeit

- A beágyazott navigátor képernyői nem kapják meg automatikusan a szülő navigátor eseményeit (például `tabPress`).
- Ha szeretnél figyelni a szülő eseményeire, használhatod a `navigation.getParent` metódust.
- Itt a `MyTabs` a szülő navigátor azonosítója, amelynek az eseményeire figyelni szeretnél.

```
const unsubscribe = navigation
  .getParent('MyTabs')
  .addListener('tabPress', (e) => {
    // Do something
    alert('Tab pressed!');
  });
```



6) A beágyazott navigátorok nem kapják meg a szülő eseményeit - Példa

Instagram tab újranyomás



- Instagramnál ha rákattintasz a Home tabra, miközben már Home-ban vagy, akkor az app visszagörget a feed tetejére.
- Ez egy tabPress event.
- De ha a Home tabon belül van egy nested stack:
 - Feed
 - Post
 - Comments
- akkor a Comments screen nem kapja meg automatikusan a tabPress eseményt.
- Ezért kell a `navigation.getParent()`



7) A szülő navigátor felülete a gyermek navigátor fölött jelenik meg

- A szülő navigátor felhasználói felülete mindig a gyermek navigátor fölött renderelődik.
- Például:
 - ha egy drawer navigátor van egy stack navigátor belsejében, akkor a drawer a stack fejléc alatt jelenik meg
 - ha egy stack navigátor van egy drawer navigátor belsejében, akkor a stack a drawer alatt jelenik meg



7) A szülő navigátor felülete a gyermek navigátor fölött jelenik meg

Gyakori minták

- Tab navigátor a stack navigátor kezdő képernyőjében → Az új képernyők elfedik a tab bart, amikor push-oljuk őket.
- Drawer navigátor a stack navigátor kezdő képernyőjében, a kezdő képernyő stack fejlécének elrejtésével → A drawer csak a stack első képernyőjéről nyitható meg.
- Stack navigátorok a drawer navigátor minden képernyőjén belül → A drawer a stack fejléc fölött jelenik meg.
- Stack navigátorok a tab navigátor minden képernyőjén belül → A tab bar mindig látható marad. Általában a tab újbóli megnyomása a stacket visszaviszi a legfelső képernyőre.

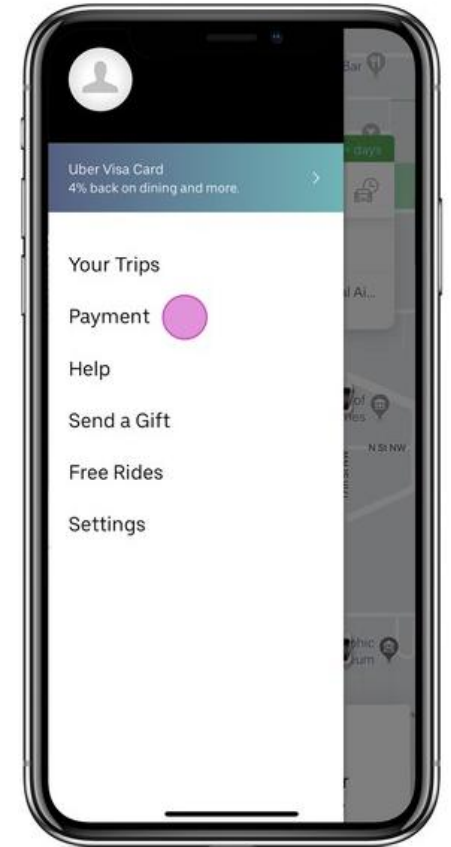


7) A szülő navigátor felülete a gyermek navigátor fölött jelenik meg - Példa

Uber

- Uber navigáció:
 - Stack
 - ❑ Home (map)
 - ❑ Ride details
- A Home screenen van egy **drawer menu**.
- Ha drawer → stack:
 - Drawer
 - ❑ Home
 - Stack
 - ❖ Map
 - ❖ Ride details
- akkor a **drawer az egész app fölött csúszik ki**.

Uber



- Ha viszont:
 - Stack
 - ❑ Drawer
- akkor a **stack header takarja a drawer-t**.
- Ezért fontos a hierarchia.



Navigálás egy beágyazott navigátor képernyőjére

Nézzük meg a következő példát (kód a képen):

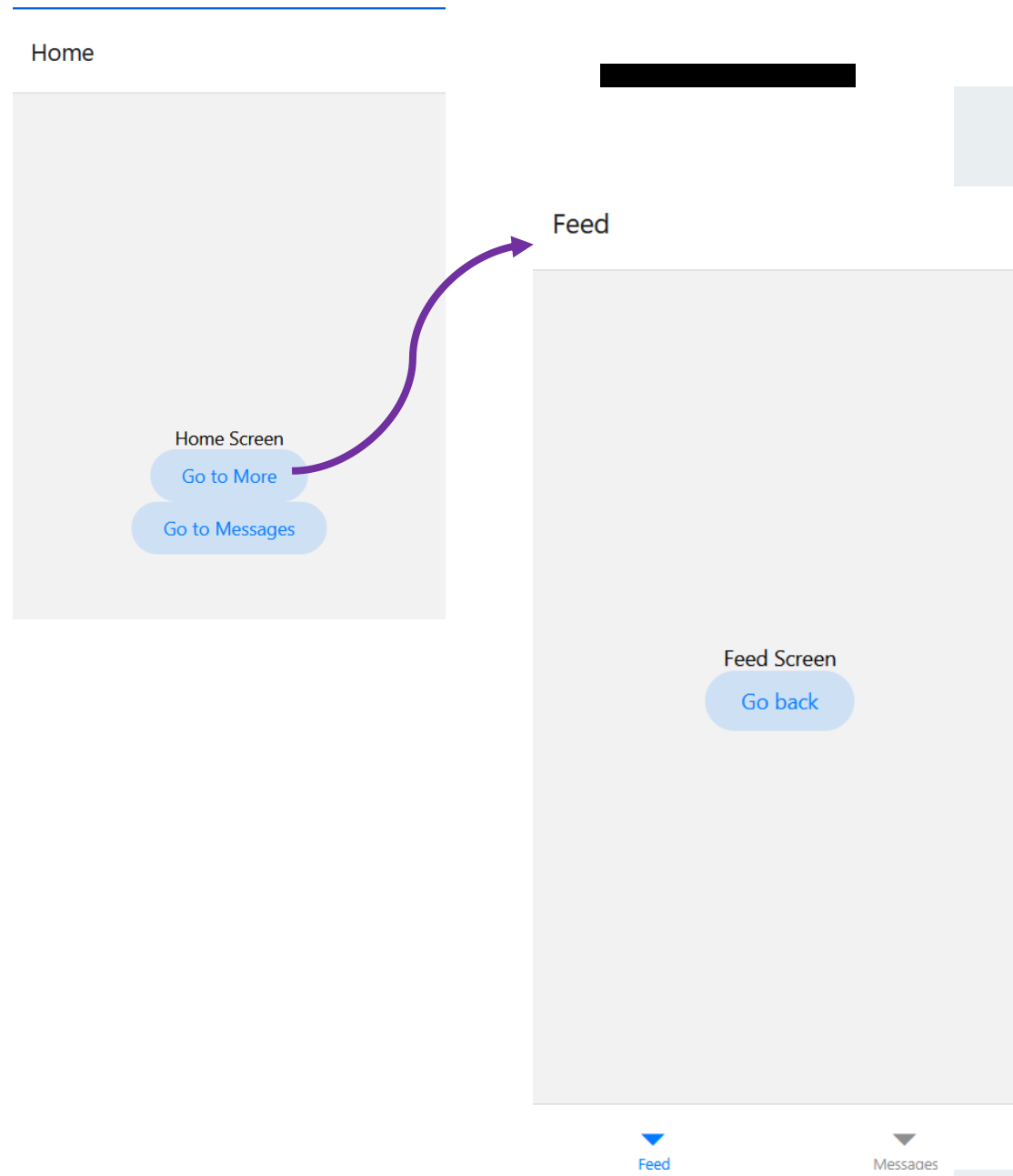
```
const MoreTabs = createBottomTabNavigator({
  screens: {
    Feed: FeedScreen,
    Messages: MessagesScreen,
  },
});

const RootStack = createNativeStackNavigator({
  screens: {
    Home: HomeScreen,
    More: {
      screen: MoreTabs,
      options: {
        headerShown: false,
      },
    },
  },
});
```

Navigálás egy beágyazott navigátor képernyőjére

- Ha a **More** képernyőre szeretnél navigálni (amely tartalmazza a **MoreTabs** navigátort) a **HomeScreen**-ről: `navigation.navigate('More');`
- Ez a **MoreTabs** navigátor kezdő képernyőjét fogja megjeleníteni (ebben az esetben a Feed képernyőt).

```
return (
  <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
    <Text>Home Screen</Text>
    <Button onPress={() => navigation.navigate('More')}>Go to More</Button>
    <Button
      onPress={() => navigation.navigate('More', { screen: 'Messages' })}
    >
      Go to Messages
    </Button>
  </View>
);
```



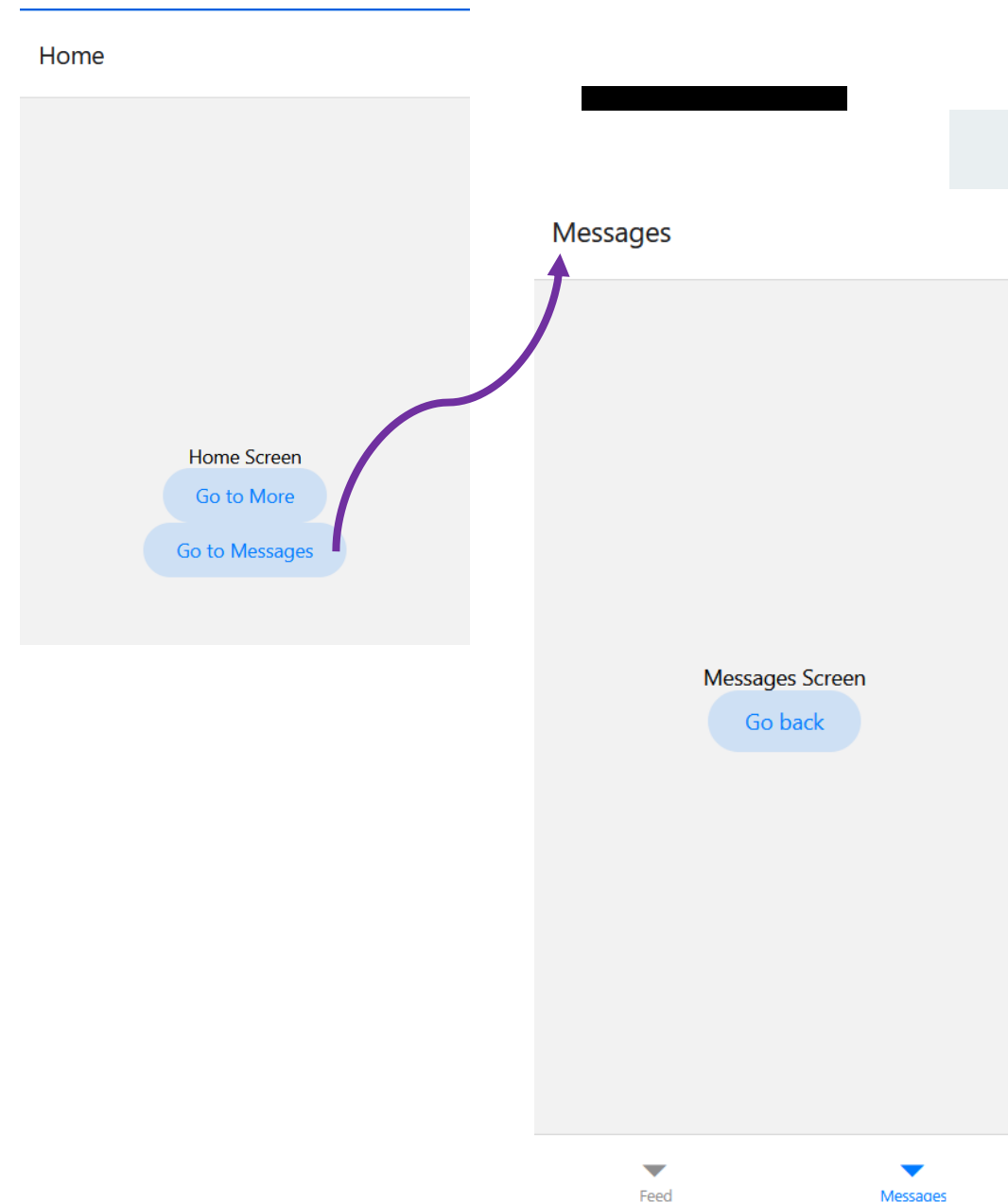
Navigálás egy beágyazott navigátor képernyőjére

- Ha egy konkrét képernyőre szeretnél navigálni a beágyazott navigátoron belül, add meg a képernyő nevét a paraméterek között:

```
navigation.navigate('More', { screen: 'Messages' });
```

- Ebben az esetben a Feed helyett a Messages képernyő fog megjelenni.

```
return (  
  <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>  
    <Text>Home Screen</Text>  
    <Button onPress={() => navigation.navigate('More')}>Go to More</Button>  
    <Button  
      |   onPress={() => navigation.navigate('More', { screen: 'Messages' })}  
    >  
      |   Go to Messages  
    </Button>  
  </View>  
)  
);
```



Paraméterek átadása egy beágyazott navigátor képernyőjének

- Paramétereket is átadhatsz a **params** kulcs megadásával:

```
navigation.navigate('More', {  
  screen: 'Messages',  
  params: { user: 'jane' },  
})
```

- Ha a navigátor már renderelve lett, akkor egy másik képernyőre történő navigálás új képernyőt fog a stack tetejére tenni (stack navigátor esetén).

Messages

Messages Screen
User: jane

Go back

▼
Feed

▼
Messages

Paraméterek átadása egy beágyazott navigátor képernyőjének

- Ugyanezt a megközelítést használhatod mélyen egymásba ágyazott képernyők esetén is.
- Fontos megjegyezni, hogy a navigate második paramétere itt egyszerűen egy params objektum, ezért például ilyesmit is csinálhatsz:

```
navigation.navigate('Home', {  
  screen: 'Settings',  
  params: {  
    screen: 'Sound',  
    params: {  
      screen: 'Media',  
    },  
  },  
});
```

Itt a Media képernyőre navigálunk, amely egy olyan navigátorban található, ami a Sound képernyőn belül van, ami pedig egy navigátor része a Settings képernyőn belül.



Paraméterek átadása egy beágyazott navigátor képernyőjének

- A screen és a hozzá tartozó paraméterek belső használatra vannak fenntartva, és a React Navigation kezeli őket.
- Bár a szülő képernyőkön elérheted például a `route.params.screen` értékét, nem ajánlott ezekre építeni, mert váratlan működéshez vezethet.

Mit jelent ez?

- Amikor ezt írod:
- A screen és params kulcsokat a React Navigation használja arra, hogy kitalálja:

```
navigation.navigate('Home', {  
  screen: 'Settings',  
  params: {  
    screen: 'Sound',  
    params: {  
      screen: 'Media',  
    },  
  },  
});
```

- melyik beágyazott navigátorba kell belépni
- azon belül melyik képernyőt kell megnyitni
- és annak milyen paramétereket kell átadni

Tehát ez valójában egy útvonal-leírás a navigátor számára.
Nem arra van tervezve, hogy az app logikája használja.



A navigátorban definiált kezdő útvonal renderelése

- Alapértelmezés szerint, amikor egy beágyazott navigátor egyik képernyőjére navigálsz, akkor a megadott képernyő lesz a kezdő képernyő, és a navigátoron beállított `initialRouteName` tulajdonság figyelmen kívül lesz hagyva.
- Ha azt szeretnéd, hogy a navigátor a saját `initialRouteName` beállítását használja, akkor kikapcsolhatod azt a viselkedést, hogy a megadott képernyő legyen a kezdő képernyő.
- Ehhez állítsd az `initial` értékét `false`-ra:

```
navigation.navigate('Root', {  
  screen: 'Settings',  
  initial: false,  
});
```

- Ez azt is befolyásolja, mi történik a vissza gomb megnyomásakor.
- Ha van egy kezdő képernyő (*initial screen*), akkor a vissza gomb oda fogja visszavinni a felhasználót.



Többszörös fejléc elkerülése egymásba ágyazott navigátoroknál

- Amikor navigátorokat egymásba ágyazol, előfordulhat, hogy mind a szülő, mind a gyermek navigátor fejlécét látod.
- Ha csak a gyermek navigátor fejlécét szeretnéd megjeleníteni, akkor állítsd a szülő képernyőn a **headerShown: false** opciót:

```
const HomeTabs = createBottomTabNavigator({
  screens: {
    Feed: FeedScreen,
    Messages: MessagesScreen,
  },
});

const RootStack = createNativeStackNavigator({
  screens: {
    Home: {
      screen: HomeTabs,
      options: {
        headerShown: false,
      },
    },
    Profile: ProfileScreen,
  },
});
```

Feed

Ha a **headerShown: true**,
akkor a Feed felett
megjelenik a Home.

Feed Screen

Go to Profile

▼
Feed

▼
Messages



Többszörös fejléc elkerülése egymásba ágyazott navigátoroknál

- Ez a beállítás a navigátorok ágyazási struktúrájától függetlenül működik.
- Ha egyik navigátorban sem szeretnél fejléctet, akkor mindegyiknél add meg a `headerShown: false`

```
const HomeTabs = createBottomTabNavigator({
  screenOptions: {
    headerShown: false,
  },
  screens: {
    Feed: FeedScreen,
    Messages: MessagesScreen,
  },
});

const RootStack = createStackNavigator({
  screenOptions: {
    headerShown: false,
  },
  screens: {
    Home: HomeTabs,
    Profile: ProfileScreen,
  },
});
```



Bevált gyakorlatok navigátorok beágyazásakor

- Próbáld a navigátorok egymásba ágyazását a lehető legkevesebbre csökkenteni, mert több hátránya is lehet:
 - A túl mélyen egymásba ágyazott nézetstruktúra memória- és teljesítményproblémákat okozhat gyengébb eszközökön.
 - Az azonos típusú navigátorok egymásba ágyazása (például tab a tabban, drawer a drawerben) zavaró felhasználói élményhez vezethet.
 - Túl sok egymásba ágyazás esetén a kód nehezen követhetővé válik, különösen amikor beágyazott képernyőkre navigálsz vagy deep linkinget konfigurálsz.
- Fontos, hogy a navigátorok egymásba ágyazását ne a kód rendszerezésére, hanem a kívánt felhasználói felület elérésére használd.



Bevált gyakorlatok navigátorok beágyazásakor

- Ha csak a képernyőket szeretnéd logikailag csoportosítani, akkor inkább a Group komponenst használd dinamikus konfigurációhoz, vagy a groups tulajdonságot statikus konfigurációhoz.

```
const MyStack = createStackNavigator({
  screens: {
    // Common screens
  },
  groups: {
    // Common modal screens
    Modal: {
      screenOptions: {
        presentation: 'modal',
      },
      screens: {
        Help,
        Invite,
      },
    },
    // Screens for logged in users
    User: {
      if: useIsLoggedIn,
      screens: {
        Home,
        Profile,
      },
    },
    // Auth screens
    Guest: {
      if: useIsGuest,
      screens: {
        SignIn,
        SignUp,
      },
    },
  },
});
```



Navigációs életrciklus (Navigation lifecycle)



- Ha webes háttérből érkezel, valószínűleg azt várnád, hogy amikor az A útvonalról a B útvonalra navigálsz, akkor az A komponens unmountolódik, majd amikor visszatérsz rá, újra mountolódik.
- A React Navigation azonban másképp működik - ezt a mobil navigáció összetettebb igényei indokolják.
- A webes böngészőkkel ellentétben a React Navigation nem unmountolja a képernyőket, amikor elnavigálsz róluk.
- Amikor a Home képernyőről a Profile képernyőre navigálsz:
 - a Profile képernyő mountolódik
 - a Home képernyő mountolva marad
- Amikor visszánavigálsz a Profile képernyőről a Home képernyőre:
 - a Profile képernyő unmountolódik
 - a Home képernyő nem mountolódik újra, hanem a már létező példánya jelenik meg



Navigációs életrciklus (Navigation lifecycle)

- Hasonló viselkedés figyelhető meg (különböző kombinációkban) más navigátorok esetében is.
- Vegyünk például egy tab navigátort két tabbal, ahol mindkét tab egy stack navigátort tartalmaz:

```
const SettingsStack = createNativeStackNavigator({
  screens: {
    Settings: SettingsScreen,
    Profile: ProfileScreen,
  },
});

const HomeStack = createNativeStackNavigator({
  screens: {
    Home: HomeScreen,
    Details: DetailsScreen,
  },
});

const MyTabs = createBottomTabNavigator({
  screenOptions: {
    headerShown: false,
  },
  screens: {
    First: SettingsStack,
    Second: HomeStack,
  },
});
```

- Kezdetben a HomeScreen képernyőn vagyunk, majd navigálunk a DetailsScreen képernyőre.
- Ezután a tab bart használva átváltunk a SettingsScreen képernyőre, majd onnan a ProfileScreen képernyőre navigálunk.
- Miután ez a művelet sor lefutott, mind a négy képernyő mountolva van.
- Ha ezután a tab bart használva visszaváltasz a HomeStack navigátorra, azt fogod látni, hogy a DetailsScreen jelenik meg - vagyis a HomeStack navigációs állapota megmaradt.

További tudnivaló a Navigation lifecycle-ről:
<https://reactnavigation.org/docs/navigation-lifecycle/>



Próbáljuk ki a bottom Tab-on keresztül a beágyazott navigációs megoldásokat

